

Exploratory Analysis of Rainfall Data in India for Agriculture

Project Documentation

Team ID: LTVIP2026TMIDS61292

Team Size: 4

Team Leader: Mohammed Owais Pateghar

Team Member: Chakali Chandrashekhar

Team Member: Pasumarthi Meghana

Team Member: Nasaka Likhitha

Abstract / Project Overview

This project focuses on predicting rainfall using machine learning techniques based on historical weather data. Exploratory Data Analysis is performed to understand patterns and relationships between meteorological features. A Random Forest Classifier is trained after preprocessing the data by handling missing values, encoding, and feature scaling. The trained model is deployed using a Flask web application for real-time prediction. The system provides rainfall prediction results along with probability percentages to support informed decision-making.

Objectives

The key objectives of this project are:

- To analyze and preprocess historical rainfall and weather data using exploratory data analysis techniques.
- To develop and evaluate a machine learning model for accurate rainfall prediction.
- To deploy the trained model using a Flask web application that displays rainfall prediction results with probability percentages.

Technologies Used (Python, Libraries, etc.)

The project utilizes Python programming language along with several essential libraries for data handling and visualization:

- Python: Core programming language used for analysis.

- NumPy: For numerical computations and array manipulation.
- Pandas: For data cleaning, transformation, and exploration.
- Matplotlib: For basic plotting and charting.
- Seaborn: For advanced and aesthetically appealing statistical visualizations.
- Scikit-learn: For preprocessing tasks such as scaling and encoding.

Dataset Description

The dataset used in this project contains historical weather observations collected over several years. It includes both numerical and categorical features such as temperature, humidity, pressure, and rainfall. The data consists of more than 140,000 records from different locations. The target variable RainTomorrow indicates whether rainfall is expected on the next day. This dataset provides sufficient information for analyzing rainfall patterns and building a predictive model.

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir	WindGustSpeed	WindDir9am	...	Humidity3pm	Pressure9am
0	2008-12-01	Albury	13.4	22.9	0.6	NaN	NaN	W	44.0	W	...	22.0	1007.7
1	2008-12-02	Albury	7.4	25.1	0.0	NaN	NaN	WNW	44.0	NNW	...	25.0	1010.6
2	2008-12-03	Albury	12.9	25.7	0.0	NaN	NaN	WSW	46.0	W	...	30.0	1007.6
3	2008-12-04	Albury	9.2	28.0	0.0	NaN	NaN	NE	24.0	SE	...	16.0	1017.6
4	2008-12-05	Albury	17.5	32.3	1.0	NaN	NaN	W	41.0	ENE	...	33.0	1010.8

5 rows × 24 columns

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 142193 entries, 0 to 142192
Data columns (total 24 columns):
#   Column              Non-Null Count  Dtype
--  --
0   Date                142193 non-null object
1   Location            142193 non-null object
2   MinTemp             141556 non-null float64
3   MaxTemp             141871 non-null float64
4   Rainfall            140787 non-null float64
5   Evaporation         81350 non-null float64
6   Sunshine            74377 non-null float64
7   WindGustDir         132863 non-null object
8   WindGustSpeed       132923 non-null float64
9   WindDir9am         132180 non-null object
10  WindDir3pm          138415 non-null object
11  WindSpeed9am        140845 non-null float64
12  WindSpeed3pm        139563 non-null float64
13  Humidity9am         140419 non-null float64
14  Humidity3pm         138583 non-null float64
15  Pressure9am         128179 non-null float64
16  Pressure3pm         128212 non-null float64
17  Cloud9am            88536 non-null float64
18  Cloud3pm            85099 non-null float64
19  Temp9am             141289 non-null float64
...
22  RISK_MM             142193 non-null float64
23  RainTomorrow        142193 non-null object
dtypes: float64(17), object(7)
memory usage: 26.0+ MB
```

The dataset consists of more than 140,000 records with multiple numerical and categorical attributes, including missing values.

Data Preprocessing

Missing values in numerical features were handled using the mean strategy, while categorical features were filled with the most frequent values. Relevant features were selected to match the user input requirements. Categorical variables were encoded using label encoding techniques. Numerical features were scaled using standardization to improve model performance.

Importing Libraries

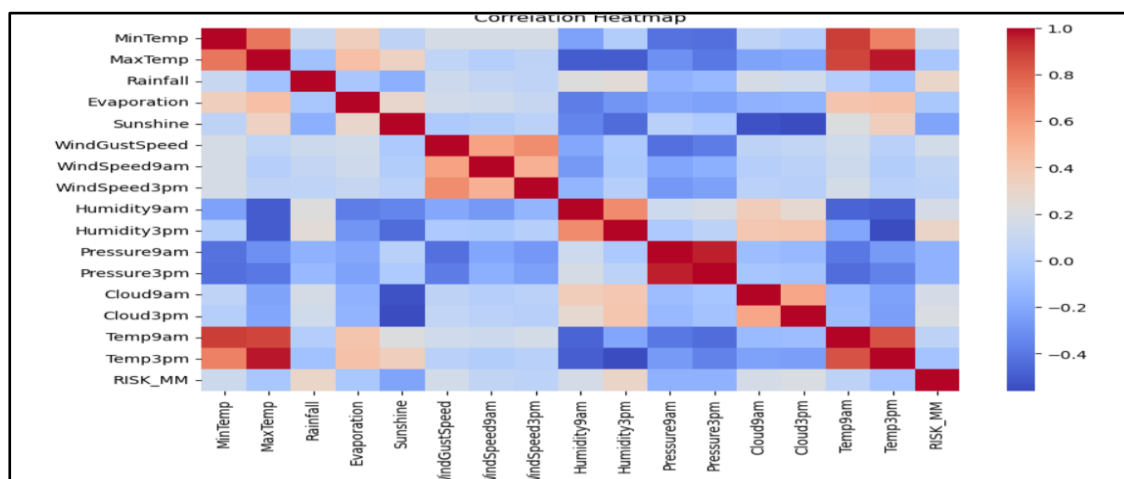
The primary Python libraries used in this project include Pandas, NumPy, Matplotlib, and Seaborn. These libraries support data manipulation, numerical computation, and visualization respectively.

Handling Missing Values

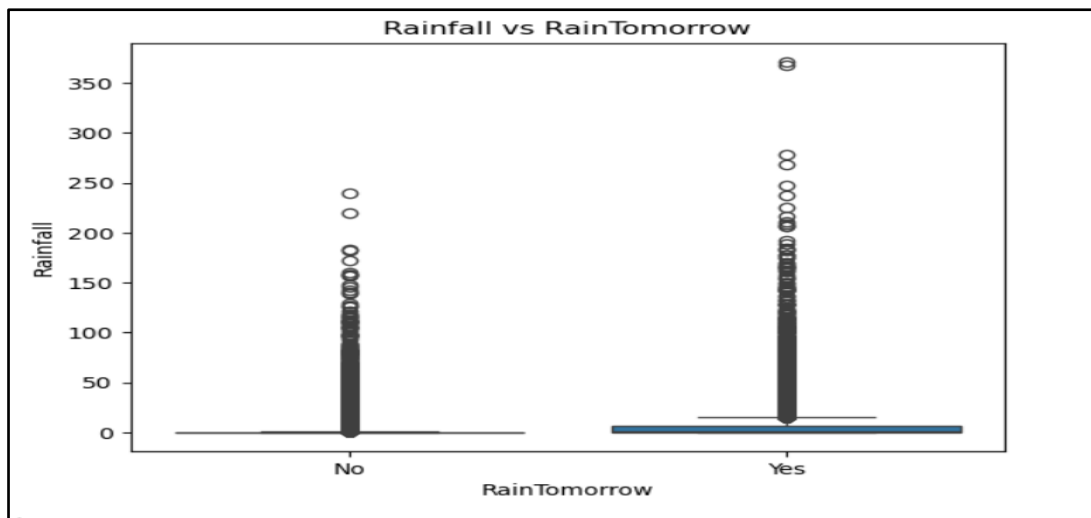
After loading the dataset, missing values were identified and handled appropriately. For numerical columns, missing values were replaced with the mean, while for categorical columns, the most frequent category was used. A missing data heatmap was also visualized to identify null value patterns.

Missing Data Visualization using Heatmap:

Missing data visualization helps in quickly identifying columns with null values and understanding their distribution across the dataset. This step ensures appropriate preprocessing techniques are applied before model training.



A heatmap visualization was used to identify the presence and distribution of missing values in the dataset. The heatmap clearly highlighted columns with missing data, allowing effective planning for data imputation.



The box plot compares the distribution of rainfall values for rainy and non-rainy days.

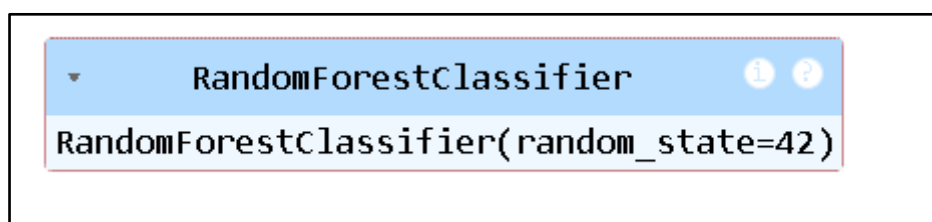
It shows that days with higher rainfall values are more likely to result in rain on the following day. The plot also highlights the presence of outliers in rainfall data.

Encoding and Scaling

Categorical features were converted into numerical form using encoding techniques. Feature scaling was applied using StandardScaler to normalize numerical features, ensuring uniform contribution of each feature during analysis.

Model Building

A Random Forest Classifier was selected and initialized to build the rainfall prediction model. This algorithm was chosen due to its robustness, ability to handle non-linear data, and high accuracy in classification tasks. The model was prepared for training using the preprocessed dataset.



Random Forest combines multiple decision trees to improve prediction accuracy and reduce overfitting, making it suitable for rainfall prediction.

Model Training

The Random Forest Classifier was trained using the training dataset obtained after the train-test split. During training, the model learned patterns and relationships between weather features and rainfall occurrence.

Model Evaluation

The trained Random Forest model was evaluated using standard classification metrics to measure its performance. The evaluation included accuracy score, confusion matrix, and classification report. The model achieved an accuracy of approximately 84%, indicating good predictive performance for rainfall classification.

Accuracy: 0.8419775660184957					
Confusion Matrix:					
[[20927 1171]					
[3323 3018]]					
Classification Report:					
	precision	recall	f1-score	support	
0	0.86	0.95	0.90	22098	
1	0.72	0.48	0.57	6341	
accuracy			0.84	28439	
macro avg	0.79	0.71	0.74	28439	
weighted avg	0.83	0.84	0.83	28439	

Model evaluation helps assess how well the trained model performs on unseen data. The accuracy score and confusion matrix confirm that the Random Forest model predicts rainfall effectively.

Saving the Model

After successful training and evaluation, the trained Random Forest model and preprocessing objects were saved using the Pickle library. Saving the model allows it to be reused later without retraining and ensures consistency during deployment. The saved files were later loaded into the Flask application for real-time rainfall prediction.

Saved Files Include:

rainfall.pkl – Trained Random Forest model

scale.pkl – StandardScaler used for feature scaling

encoder.pkl – Label encoder for categorical data

imputer.pkl – Imputer used for handling missing values

Implementation

The rainfall prediction system was implemented by integrating the trained Random Forest model with a Flask web application. The saved model and preprocessing files were loaded into the Flask backend. An HTML-based user interface was developed to collect

weather parameters from the user. The backend processes the input data, applies scaling, and performs prediction using the trained model. The system dynamically routes the user to appropriate result pages based on the prediction.

```
✓ TERMINAL
PS D:\Rainfall Prediction> python app.py
* Serving Flask app 'app'
* Debug mode: on
• WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 352-244-626
```

This output verifies that the backend server is active and ready to accept user requests for rainfall prediction through the web interface.

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:5000'. The main content area is titled 'Rainfall Prediction System'. It contains a series of input fields for the following parameters: Min Temperature (°C), Max Temperature (°C), Rainfall (mm), Humidity at 9 AM (%), Humidity at 3 PM (%), Pressure at 9 AM (hPa), Pressure at 3 PM (hPa), Temperature at 9 AM (°C), Temperature at 3 PM (°C), and Rain Today (a dropdown menu with 'Yes' selected). At the bottom of these fields is a blue button labeled 'Predict Rainfall'. The interface is styled with a teal bar on the left and a blue bar on the right.

This interface enables users to easily interact with the rainfall prediction system by providing necessary inputs for real-time prediction.

Result

After submitting the weather parameters, the system processes the input data and predicts the likelihood of rainfall for the next day. The result is displayed on a separate output page showing rainfall prediction chances in percentage (%), indicating whether rain is expected or not.

 **Rainfall Prediction System**

Min Temperature (°C)

Max Temperature (°C)

Rainfall (mm)

Humidity at 9 AM (%)

Humidity at 3 PM (%)

Pressure at 9 AM (hPa)

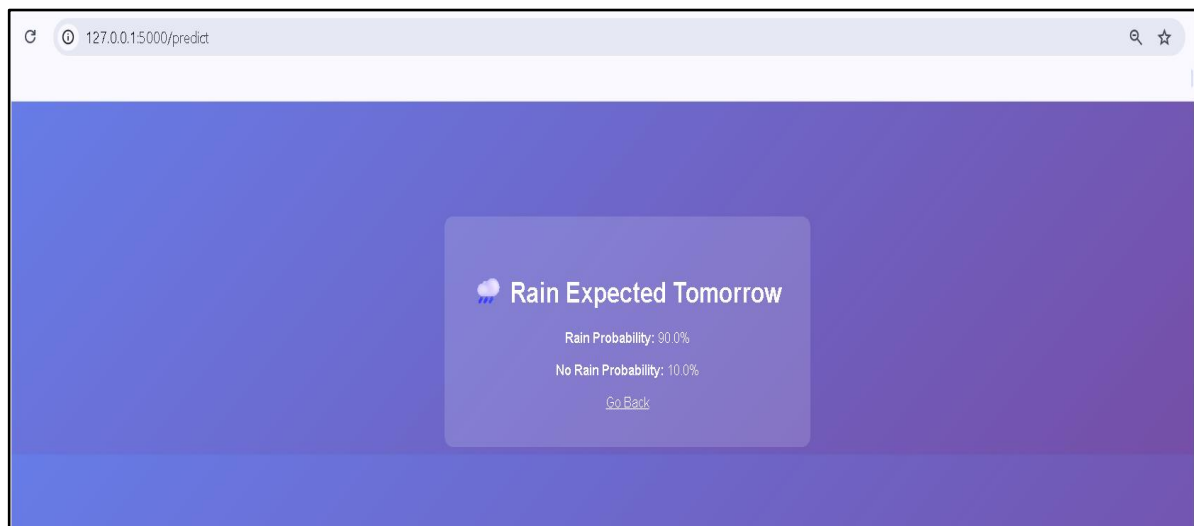
Pressure at 3 PM (hPa)

Temperature at 9 AM (°C)

Temperature at 3 PM (°C)

Rain Today

Predict Rainfall



The figures shows the user entering real-time weather parameters such as temperature, rainfall, humidity, pressure and there is 90% chance for rain.



Rainfall Prediction System

Min Temperature (°C)

Max Temperature (°C)

Rainfall (mm)

Humidity at 9 AM (%)

Humidity at 3 PM (%)

Pressure at 9 AM (hPa)

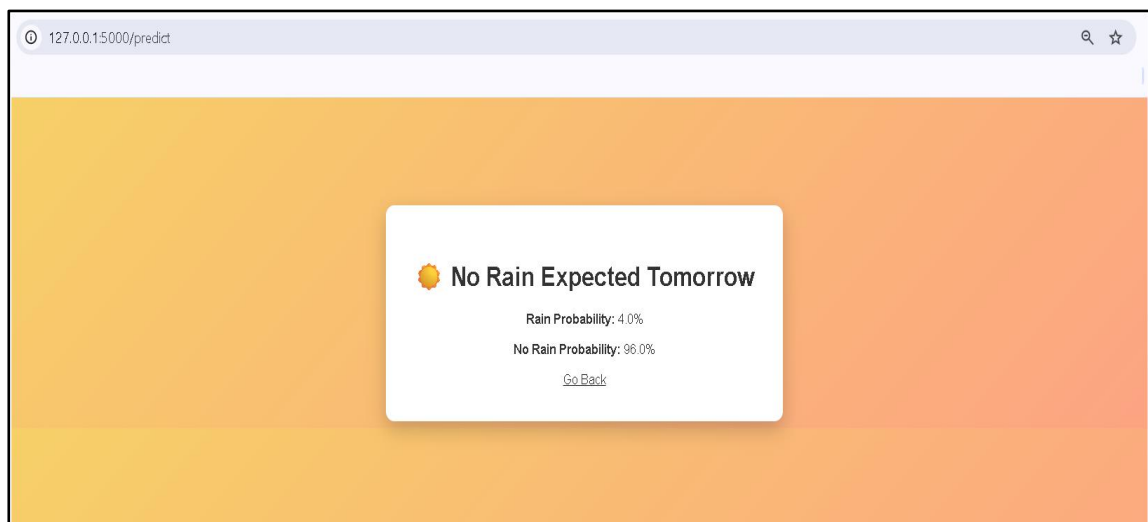
Pressure at 3 PM (hPa)

Temperature at 9 AM (°C)

Temperature at 3 PM (°C)

Rain Today

[Predict Rainfall](#)



The figures display the prediction outcome indicating that no rainfall is expected tomorrow, with a rain probability of 4% and a no-rain probability of 96%, as generated by the trained machine learning model.

Future Scope

- The system can be enhanced by integrating real-time weather data APIs to provide live rainfall predictions.
- Advanced models such as XGBoost, LSTM, or deep learning techniques can be used to improve prediction accuracy.
- The application can be extended to support location-based forecasting and mobile platforms for wider accessibility.

Conclusion

The Rainfall Prediction System successfully analyzes historical weather data and predicts rainfall occurrence using machine learning techniques. By integrating a trained model with a Flask-based web application, the system provides probability-based rainfall predictions through a user-friendly interface. This project demonstrates the effective use of data preprocessing, model building, and web deployment to support weather-based decision-making.

Appendix

Project Structure:

app.py – The Flask backend file used for server-side scripting and application building.
templates/ – The folder containing all the HTML templates including index.html, chance.html, and noChance.html.

Rainfall.pkl – The Trained ML Model file used for making predictions in the web application.

scale.pkl – A saved file used for scaling the input data. encoder.pkl

– A saved file used for encoding categorical data. imputer.pkl – A saved file used for imputing (filling) missing values.

rainfall_prediction.ipynb – The model training notebook file

References

- Flask documentation – <https://flask.palletsprojects.com/>
- Scikit-learn – <https://scikit-learn.org/>
- Python official documentation – <https://docs.python.org/3/>

Demonstration

This section provides visual evidence of the developed project, including source code

snippets and the web application interface.

app.py

```
File Edit Selection View Go Run Terminal Help ← → Rainfall_Prediction

EXPLORER
  RAINFALL_PREDICTION
    templates
      chance.html
      index.html
      noChance.html
    app.py
    encoder.pkl
    imputer.pkl
    Rainfall_prediction.ipynb
    rainfall.pkl
    scale.pkl
    weatherAUS.csv

app.py
1 from flask import Flask, render_template, request
2 import numpy as np
3 import pandas as pd
4 import pickle
5
6 app = Flask(__name__)
7
8 # Load saved model and preprocessing objects
9 model = pickle.load(open('rainfall.pkl', 'rb'))
10 scaler = pickle.load(open('scale.pkl', 'rb'))
11 encoder = pickle.load(open('encoder.pkl', 'rb'))
12 imputer = pickle.load(open('imputer.pkl', 'rb'))
13
14 @app.route('/')
15 def home():
16     return render_template('index.html')
17
18 @app.route('/predict', methods=['POST'])
19 def predict():
20     # Get inputs from form
21     input_data = [
22         float(request.form['MinTemp']),
23         float(request.form['MaxTemp']),
24         float(request.form['Rainfall']),
25         float(request.form['Humidity9am']),
26         float(request.form['Humidity3pm']),
27         float(request.form['Pressure9am']),
28         float(request.form['Pressure3pm']),
29         float(request.form['Temp9am']),
30         float(request.form['Temp3pm']),
31         int(request.form['RainToday'])
32     ]
```

```
File Edit Selection View Go Run Terminal Help ← → Rainfall_Prediction

EXPLORER
  RAINFALL_PREDICTION
    templates
      chance.html
      index.html
      noChance.html
    app.py
    encoder.pkl
    imputer.pkl
    Rainfall_prediction.ipynb
    rainfall.pkl
    scale.pkl
    weatherAUS.csv

app.py
19 def predict():
33
34     columns = [
35         'MinTemp', 'MaxTemp', 'Rainfall',
36         'Humidity9am', 'Humidity3pm',
37         'Pressure9am', 'Pressure3pm',
38         'Temp9am', 'Temp3pm',
39         'RainToday'
40     ]
41
42     df = pd.DataFrame([input_data], columns=columns)
43
44     # Scale data
45     df_scaled = scaler.transform(df)
46
47     # Get prediction probabilities
48     probs = model.predict_proba(df_scaled)[0]
49     no_rain_prob = round(probs[0] * 100, 2)
50     rain_prob = round(probs[1] * 100, 2)
51
52     # Render result
53     if rain_prob >= 50:
54         return render_template(
55             'chance.html',
56             rain_prob=rain_prob,
57             no_rain_prob=no_rain_prob
58         )
59     else:
60         return render_template(
61             'noChance.html',
62             rain_prob=rain_prob,
63             no_rain_prob=no_rain_prob
64         )
65
66 if __name__ == "__main__":
67     app.run(debug=True)
```

Rainfall_prediction.ipynb

```
app.py Rainfall_prediction.ipynb X
Rainfall_prediction.ipynb > import numpy as np
Generate + Code + Markdown | Run All Restart Clear All Outputs | Jupyter Variables Outline ...
base (Python 3.13.5)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pickle

[1] Python

data = pd.read_csv("weatherAUS.csv")
data.head()

[2] Python

...

```

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir	WindGustSpeed	WindDir9am	...	Humidity3pm	Pressure9am
0	2008-12-01	Albury	13.4	22.9	0.6	NaN	NaN	W	44.0	W	...	22.0	1007.7
1	2008-12-02	Albury	7.4	25.1	0.0	NaN	NaN	WNW	44.0	NNW	...	25.0	1010.6
2	2008-12-03	Albury	12.9	25.7	0.0	NaN	NaN	WSW	46.0	W	...	30.0	1007.6
3	2008-12-04	Albury	9.2	28.0	0.0	NaN	NaN	NE	24.0	SE	...	16.0	1017.6
4	2008-12-05	Albury	17.5	32.3	1.0	NaN	NaN	W	41.0	ENE	...	33.0	1010.8

5 rows x 24 columns

```
app.py Rainfall_prediction.ipynb X
Rainfall_prediction.ipynb > import numpy as np
Generate + Code + Markdown | Run All Restart Clear All Outputs | Jupyter Variables Outline ...

data.info()
data.describe()
data.shape

[3] Python

...
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 142193 entries, 0 to 142192
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Date                  142193 non-null object
1   Location              142193 non-null object
2   MinTemp               141556 non-null float64
3   MaxTemp               141871 non-null float64
4   Rainfall              140787 non-null float64
5   Evaporation           81350 non-null float64
6   Sunshine              74377 non-null float64
7   WindGustDir           132863 non-null object
8   WindGustSpeed         132923 non-null float64
9   WindDir9am            132180 non-null object
10  WindDir3pm            138415 non-null object
11  WindSpeed9am          140845 non-null float64
12  WindSpeed3pm          139563 non-null float64
13  Humidity9am           140419 non-null float64
14  Humidity3pm           138583 non-null float64
15  Pressure9am           128179 non-null float64
16  Pressure3pm           128212 non-null float64
17  Cloud9am              88536 non-null float64
18  Cloud3pm              85099 non-null float64
19  Temp9am               141289 non-null float64
...
22  RISK_MM               142193 non-null float64
23  RainTomorrow          142193 non-null object
dtypes: float64(17), object(7)
memory usage: 26.0+ MB

```

```
num_imputer = SimpleImputer(strategy='mean')
cat_imputer = SimpleImputer(strategy='most_frequent')

num_cols = data.select_dtypes(include=['int64', 'float64']).columns
cat_cols = data.select_dtypes(include='object').columns

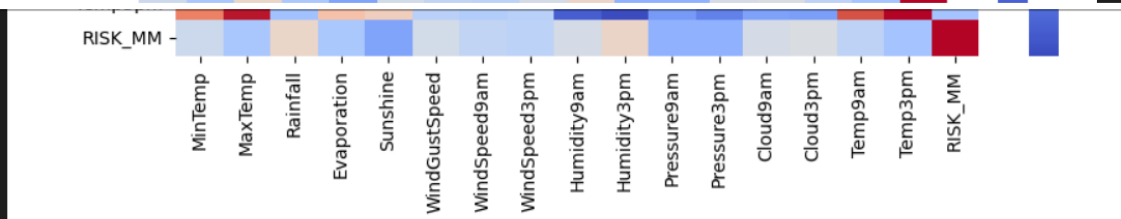
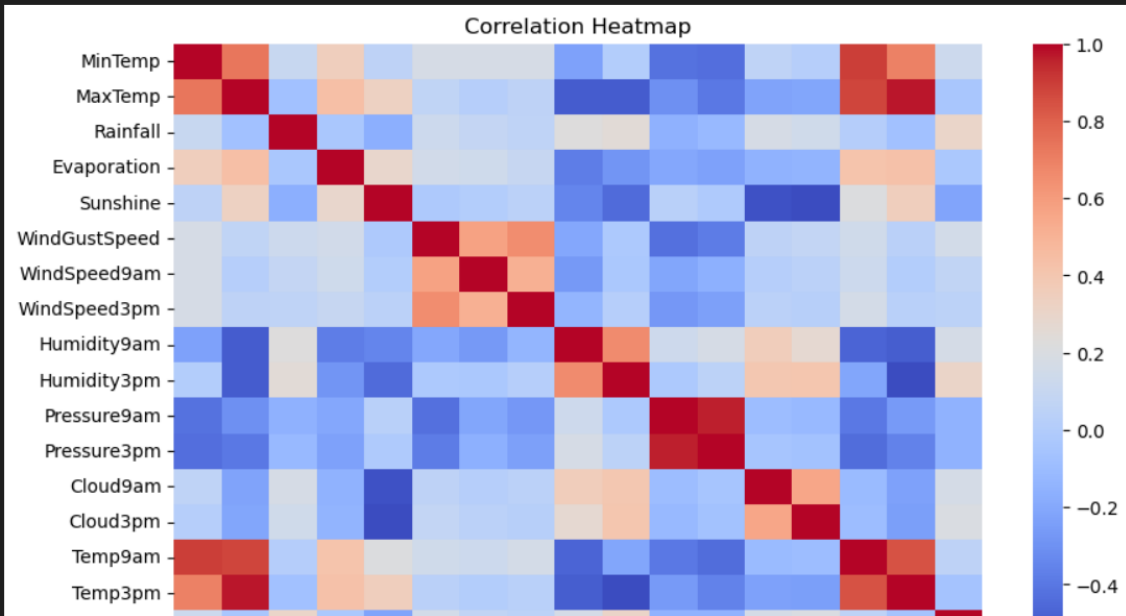
data[num_cols] = num_imputer.fit_transform(data[num_cols])
data[cat_cols] = cat_imputer.fit_transform(data[cat_cols])

plt.figure(figsize=(10,6))
numeric_data = data.select_dtypes(include=['int64', 'float64'])
sns.heatmap(numeric_data.corr(), cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

sns.boxplot(x=data['RainTomorrow'], y=data['Rainfall'])
plt.title("Rainfall vs RainTomorrow")
plt.show()
```

```
plt.figure(figsize=(10,6))
numeric_data = data.select_dtypes(include=['int64','float64'])
sns.heatmap(numeric_data.corr(), cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

sns.boxplot(x=data['RainTomorrow'], y=data['Rainfall'])
plt.title("Rainfall vs RainTomorrow")
plt.show()
```



```

final_features = [
    'MinTemp', 'MaxTemp', 'Rainfall',
    'Humidity9am', 'Humidity3pm',
    'Pressure9am', 'Pressure3pm',
    'Temp9am', 'Temp3pm',
    'RainToday'
]

X = data[final_features]
y = data['RainTomorrow']

```

[6]



```

X = X.copy() # 🔥 ADD THIS LINE

from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
X.loc[:, 'RainToday'] = le.fit_transform(X['RainToday'])
y = le.fit_transform(y)

```

[7]

```

sc = StandardScaler()
X_scaled = sc.fit_transform(X)

```

[8]

```

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

```

[9]

```

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

```

[10]

...

RandomForestClassifier ⓘ ⓘ
RandomForestClassifier(random_state=42)

```

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

```

[11]

...

Accuracy: 0.8419775660184957

Confusion Matrix:

```

[[20927  1171]
 [ 3323  3018]]

```

Classification Report:

	precision	recall	f1-score	support
0	0.86	0.95	0.90	22098
1	0.72	0.48	0.57	6341
accuracy			0.84	28439
macro avg	0.79	0.71	0.74	28439
weighted avg	0.83	0.84	0.83	28439

```

pickle.dump(model, open('rainfall.pkl', 'wb'))
pickle.dump(sc, open('scale.pkl', 'wb'))
pickle.dump(le, open('encoder.pkl', 'wb'))
pickle.dump(num_imputer, open('imputer.pkl', 'wb'))

```

[12]

Outputs

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL JUPYTER
PORTS
No forwarded ports. Forward a port to access your locally running services over the internet.
Forward a Port

Warning: PowerShell detected that you might be using a screen reader and has disabled PSReadLine for compatibility purposes. If you want to re-enable it, run 'Import-Module PSReadLine'.

PS D:\Rainfall_Prediction> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 352-244-626
```



Rainfall Prediction System

Min Temperature (°C)

Max Temperature (°C)

Rainfall (mm)

Humidity at 9 AM (%)

Humidity at 3 PM (%)

Pressure at 9 AM (hPa)

Pressure at 3 PM (hPa)

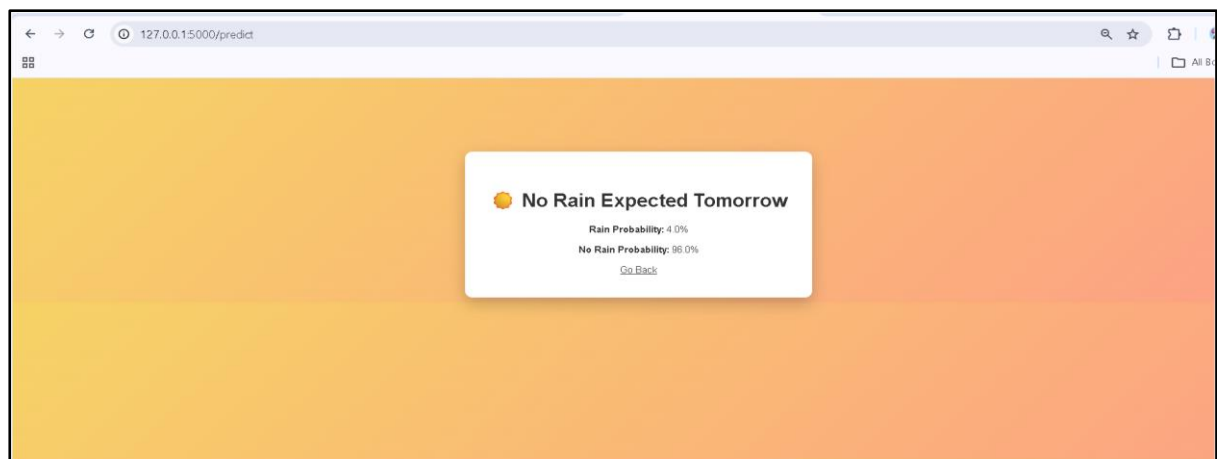
Temperature at 9 AM (°C)

Temperature at 3 PM (°C)

Rain Today

No

Predict Rainfall





Rainfall Prediction System

Min Temperature (°C)

Max Temperature (°C)

Rainfall (mm)

Humidity at 9 AM (%)

Humidity at 3 PM (%)

Pressure at 9 AM (hPa)

Pressure at 3 PM (hPa)

Temperature at 9 AM (°C)

Temperature at 3 PM (°C)

Rain Today

Predict Rainfall

